

ARTIFICIAL INTELLIGENCE

FAKE NEWS DETECTION USING NATURAL LANGUAGE PROCESSING (NLP) :

NAME : DINESHKUMAR S

NM ID : au513521104012

COLLEGE: AMCET

PHASE 4 – DEVELOPMENT PART 2 :

INTRODUCTION :

Fake news has become a significant concern in today's digital age, with the potential to spread misinformation and influence public opinion. **Natural Language Processing (NLP)** and **deep learning** techniques are powerful tools for fake news detection . In the previous documentation **AI_PHASE3** we started building the model by loading the dataset , cleaning the data and preprocessing the data .

In this phase we are going to train the model with the preprocessed data and we are going to evaluate the performance of the model and measure the accuracy of the model .

We use the fake news dataset form the Kaggle for this project . The link for the dataset is

<https://www.kaggle.com/clmentbisailon/fake-and-real-news-dataset> .

We use the **LSTM(Long Short Term Memory)** for training our models .

DEVELOPMENT STEPS FOR TRAINING THE MODEL :

1. BUILDING THE MODEL :

In this step we are going to build our model for the fake news detection. We use a sequential model for training .

```

# Sequential Model
model = Sequential()

# embeddidng layer
model.add(Embedding(total_words, output_dim = 128))
# model.add(Embedding(total_words, output_dim = 240))

# Bi-Directional RNN and LSTM
model.add(Bidirectional(LSTM(128))) # no of neurons

# Dense layers
model.add(Dense(128, activation = 'relu'))
model.add(Dense(1,activation= 'sigmoid')) # reason: we do binary
classification here
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['acc'])
model.summary()

```

We use the LSTM(Long Short Term Memory) for training the model and we specify the number of neurons we need for training the model .

Then we add the activation functions like relu (Rectified Linear Unit) and sigmoid to the model . Then get the summary of the model .

OUTPUT:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
embedding_1 (Embedding)	(None, None, 128)	13914112

bidirectional_1 (Bidirection)	(None, 256)	263168

dense_2 (Dense)	(None, 128)	32896
-----------------	-------------	-------

dense_3 (Dense)	(None, 1)	129
-----------------	-----------	-----

Total params: 14,210,305

Trainable params: 14,210,305

Non-trainable params: 0

This is the output generated for the model building .

2. **MODEL TRAINING :**

The next step after building the model is to train the model.

Here we use 2 epochs for training the model .

```
y_train = np.asarray(y_train)
```

We convert the data for training into an array using the above code .

```
model.fit(padded_train, y_train, batch_size = 64, validation_split = 0.1,  
epochs = 2)
```

This is the code for training the model . We use the padded_train that we generated during the tokenization phase in the AI_PHASE3 and the y_train to train the model .

We mention some essential parameters for training such as batch_size=64 ,validation_split=0.1 and epochs=2 .

OUTPUT:

Epoch 1/2

506/506 [=====] - 150s 297ms/step -
loss: 0.0422 - acc: 0.9829 - val_loss: 0.0090 - val_acc: 0.9983

Epoch 2/2

506/506 [=====] - 126s 248ms/step -
loss: 0.0014 - acc: 0.9997 - val_loss: 0.0114 - val_acc: 0.9978

3. ASSESS THE MODEL PERFORMANCE :

The next step after training the model is to assess the model performance .

First we make the prediction using the model.

```
pred = model.predict(padded_test)
```

If the predicted value is greater than 0.5 then it is real or else it is false .To implement this we write the below coding .

```
prediction = []  
for i in range(len(pred)):  
    if pred[i].item() > 0.5:  
        prediction.append(1)  
    else:  
        prediction.append(0)
```

Now we measure the accuracy of the model using the sklearn.metrics library .

```
from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(list(y_test), prediction)  
  
print("Model Accuracy : ", accuracy)
```

OUTPUT:

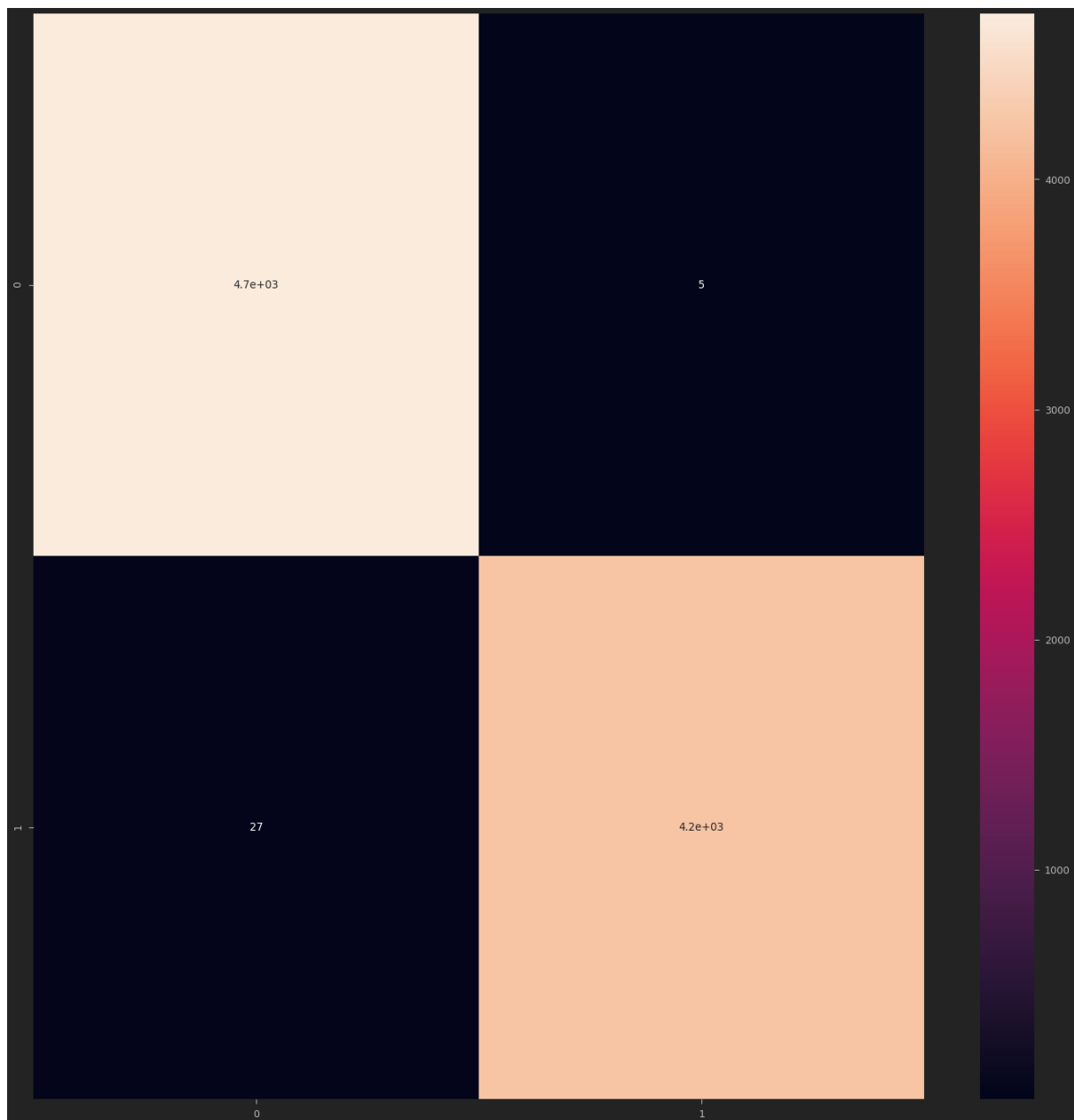
Model Accuracy : 0.9964365256124722

Thus the accuracy of our trained model is nearly 99% . If the accuracy of our model is low then we can retrain the model .

Final step is to visualize the model using the confusion matrix . We use the sklearn.metrics for this .

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(list(y_test), prediction)
plt.figure(figsize = (25, 25))
sns.heatmap(cm, annot = True)
```

OUTPUT :



This is the confusion matrix for our model . Thus the model training and measuring the accuracy of the model is finally executed successfully.

CONCLUSION :

Thus the model training and validation of the model is successfully done for the fake news detection model in this AI_PHASE4 documentation. We saw the previous steps such as loading the data , cleaning the data and preprocessing the data in AI_PHASE3 document . Thus the Fake news detection model is fully trained and well executed .